

LAPORAN TUGAS BESAR 3

IF2211 Strategi Algoritma

Disusun oleh:

LET IT RIDE - K02

Josh Reinhart Zidik - 13524048

Raysha Erviandika Putra - 13524050

Muhammad Naufal Romi Annafi - 13524058



Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Jl. Ganesha 10, Bandung 40132

BAB 1

Pendahuluan

1.1 Deskripsi Tugas

Tugas besar ini bertujuan untuk mengembangkan sebuah browser extension yang dapat mendeteksi konten judi online pada halaman web secara otomatis. Sistem bekerja dengan membaca isi DOM halaman web, kemudian melakukan proses pencocokan terhadap daftar kata kunci yang terdapat pada file `keyword.txt`.



Gambar 1: Contoh Browser Extension

```
judol
slot
slot gacor
gacor
maxwin
scatter
scatter hitam
scatter merah
jackpot
jackpot maxwin
bonus new member
bonus member baru
bonus deposit
bonus harian
bonus mingguan
bonus bulanan
bonus rolligan
```

bonus turnover . . . (408 line)
keyword.txt

Dalam implementasinya, sistem wajib menggunakan algoritma pencocokan string Knuth-Morris-Pratt (KMP) dan Boyer-Moore untuk melakukan exact matching. Selain itu, sistem juga menggunakan Regular Expression (RegEx) untuk mendeteksi pola teks tertentu seperti kombinasi kata dan angka tanpa spasi, misalnya MAXWIN234 atau SLOT99.

Apabila exact matching tidak menemukan hasil yang sesuai, sistem akan melakukan proses fuzzy matching menggunakan Weighted Levenshtein Distance untuk mengukur tingkat kemiripan kata. Pendekatan ini memungkinkan sistem tetap mendeteksi kata yang telah dimanipulasi menggunakan karakter mirip secara visual, seperti:

- Gacor999
- Gacor999
- H0KI88
- H0KI88

Sistem juga menyediakan mekanisme visualisasi berupa highlight atau blur pada elemen DOM yang terdeteksi mengandung konten judi online, serta menampilkan statistik hasil deteksi secara realtime pada popup extension.

BAB 2

Landasan Teori

2.1 Algoritma Knuth-Morris-Pratt

KMP adalah algoritma pencarian string dengan cara menghindari pergeseran pola yang tidak perlu. Algoritma ini memanfaatkan informasi dari pola itu sendiri untuk menentukan seberapa jauh pola harus digeser ketika terjadi ketidakcocokan.

Ide kuncinya adalah ketika pola tidak cocok dengan teks pada posisi tertentu, algoritma tidak langsung memulai dari awal. Sebaliknya, algoritma menggunakan failure function/border array yang menyimpan informasi tentang prefiks pola yang juga merupakan sufiks. Informasi ini memungkinkan pola untuk digeser tanpa memeriksa ulang karakter yang sudah pasti cocok.

Contohnya, misal sedang mencari pola "ABABC" dalam teks dan sudah menemukan "ABAB" tetapi karakter ke-5 tidak cocok, maka kita tahu bahwa 2 karakter pertama dari suffix ("AB") sama dengan 2 karakter pertama dari pola. Jadi kita bisa langsung melanjutkan dari sana tanpa dari awal.

2.2 Algoritma Boyer-Moore

Boyer-Moore adalah algoritma yang menggunakan pendekatan dari kanan ke kiri untuk membandingkan pola dengan teks. Ini berbeda dari kebanyakan algoritma yang membaca dari kiri ke kanan.

Ide utamanya adalah perbandingan dimulai dari karakter paling kanan pola. Ketika terjadi ketidakcocokan, pola digeser sejauh mungkin berdasarkan informasi karakter yang tidak cocok tersebut. Algoritma menggunakan dua aturan: Bad Character Rule yaitu jika karakter pada teks tidak ditemukan di pola, pola dapat digeser melewati karakter tersebut sepenuhnya serta Good Suffix Rule yaitu jika sebagian pola sudah cocok (suffix), pola dapat digeser dengan memperhitungkan sufiks yang telah cocok tersebut.

Cara kerja algoritma bayangkan sedang mencari sebuah kata di halaman buku, tapi alih-alih mengecek dari huruf pertama, kita mulai mengecek dari huruf terakhir kata yang kamu cari. Kalau huruf terakhirnya saja sudah tidak cocok dengan kata di halaman, kita langsung melompat beberapa karakter ke kanan tanpa memeriksa satu per satu (seperti menghindari bagian yang

jelas salah). Namun, kalau bagian akhir sudah cocok sebagian, kita tidak mulai dari awal lagi, kita memanfaatkan bagian yang sudah cocok itu untuk menggeser posisi pencarian ke tempat yang paling masuk akal. Dengan cara ini, kita bisa menemukan kata jauh lebih cepat tanpa memeriksa semua posisi satu per satu.

2.3 Algoritma Aho-Corasick

Aho-Corasick adalah perluasan dari KMP yang dirancang untuk mencari multiple patterns sekaligus dalam satu teks dengan efisien. Ini adalah solusi ketika kita perlu mencari banyak keyword dari sebuah dictionary dalam satu dokumen besar.

Inti dari algoritma ini adalah membangun struktur data Trie atau Prefix Tree yang menyimpan semua pola yang akan dicari. Setiap node dalam Trie merepresentasikan satu karakter dari pola. Menambahkan failure links yang mirip dengan KMP untuk menangani ketidakcocokan. Menelusuri teks hanya sekali saja, memeriksa semua pola pada waktu yang sama.

Algoritma ini seperti memiliki pohon dari semua keyword yang ingin dicari. Saat membaca teks karakter demi karakter, kita bergerak di pohon ini. Kapan pun mencapai leaf node, kita tahu keyword tersebut ditemukan.

2.4 Algoritma Rabin Karp

Rabin-Karp adalah algoritma yang menggunakan hashing untuk pattern matching. Alih-alih membandingkan karakter demi karakter, algoritma membandingkan hash values dari pola dan substring teks.

Algoritma ini bekerja dengan menghitung hash value unik dari pola yang dicari serta untuk setiap substring teks dengan ukuran sama. Lalu, dilakukan erbandingan awal dilakukan pada level hash. Jika hash cocok, baru dilakukan verifikasi karakter demi karakter untuk menghindari false positive dari hash collision. Rolling hash digunakan untuk perhitungan yang efisien saat bergeser ke substring berikutnya.

Analoginya seperti menggunakan fingerprint untuk identifikasi cepat, alih-alih membaca seluruh dokumen. Jika fingerprint cocok, baru membaca dokumen sepenuhnya untuk memastikan.

2.5 Penjelasan Browser Extension

Browser Extension merupakan aplikasi software kecil yang memperluas fungsionalitas web. Browser Extension dapat memodifikasi tampilan dan perilaku halaman web, mengakses data dari web pages, berjalan di background tanpa intervensi pengguna, serta menyimpan data lokal dan melakukan komunikasi dengan server.

Browser yang dibangun pada tugas besar ini dirancang untuk mendeteksi dan menandai keyword tidak diinginkan yang berkaitan dengan judi online di halaman web secara real-time. Browser Extension ini dibangun menggunakan berbagai algoritma string matching untuk meningkatkan akurasi deteksi.

Teknologi yang digunakan pada Browser Extension ini meliputi

- TypeScript untuk type safety
- Vite untuk bundling dan development
- Chrome Manifest V3 untuk extension
- Tesseract.js untuk OCR
- Browser APIs: Chrome Storage, Tabs, Content Scripts

BAB 3

Analisis Pemecahan Masalah

3.1 Proses Pemecahan Masalah

Masalah utama dari tugas besar ini adalah mendeteksi keberadaan keyword tertentu dalam konten halaman web secara real-time dan memberikan visual feedback kepada pengguna. Masalah ini mencakup tiga hal yaitu masalah string matching, masalah variasi karakter, dan masalah performa.

Permasalahan tersebut akhirnya dibagi menjadi beberapa tahap penyelesaian untuk masing-masing masalah:

1. Ekstraksi Data

Tahap ini pertama dilakukan untuk mengambil seluruh teks dari halaman web sehingga dapat diproses. Ekstraksi data dilakukan menggunakan TreeWalker dengan Traverse DOM Tree, lalu filter OUT beberapa tags seperti SCRIPT dan STYLE, lalu mengumpulkan semua text node yang berguna, dan terkahir simpan referensi ke node asli untuk highlight

2. Normalisasi

Raw Text yang didapatkan dari DOM pertama akan di-convert ke lowercase lalu dilakukan normalisasi menggunakan homoglyph character sehingga menyamakan representasi karakter yang secara visual mirip.

3. Exact Pattern Matching

Tahap pencocokan pertama menggunakan pencocokan sama persis untuk mencari keyword menggunakan algoritma yang dipilih dari KMP, BM, Aho-Corasick, serta Rabin-Karp. Hasil text yang didapatkan dari normalisasi diproses menggunakan algoritma lalu dihasilkan array hasil komputasi.

4. Regex Pattern Matching

Selain dilakukan exact match, dilakukan juga pattern matching menggunakan regex secara paralel. Regex dilakukan dengan menangkap pola khusus keyword dan numeric suffix yang ada dalam text.

5. Fuzzy Pattern Matching

Teknik Exact dan Regex Matching bisa saja tidak cukup karena developer web bisa saja melakukan teknik phishing yaitu membuat nama domain yang serupa dengan pada umumnya, namun nyatanya berbeda. Ini terjadi oleh karena adanya perbedaan kode komputer (unicode), dan komputer mengiranya adalah karakter yang berbeda. Hal ini menyebabkan perlu dilakukan matching menggunakan Levenshtein distance untuk setiap keyword, jika distance lebih besar daripada threshold, bagian text akan dihitung sebagai match.

6. Deduplication dan Overlap Resolution

Seluruh hasil dari pattern matching sebelumnya dihilangkan yang overlapping sehingga hanya satu match per text segment. Prioritas hasil yang match diurutkan exact > fuzzy > regex.

7. Highlighting dan Visual Feedback

Seluruh hasil text node beserta array matches digunakan untuk membangun sebuah document fragment yang berisi teks asli dan elemen <mark> pada bagian yang cocok dengan pola pencarian. Setelah fragment selesai dibuat, text node asli digantikan dengan fragment tersebut sehingga hasil pencarian dapat ditampilkan secara visual melalui proses highlighting pada DOM. Output akhir dari proses ini adalah tampilan DOM yang telah diberi penanda pada bagian teks yang sesuai.

8. OCR Processing

Selain melakukan processing untuk text dari web juga dilakukan pattern matching di elemen gambar dari halaman secara paralel sehingga pendeteksian keyword menjadi lebih bagus

3.2 Pemetaan Masalah

Berikut merupakan pemetaan masalah tugas besar ini ke elemen algoritma

1. Algoritma KMP

Elemen Masalah	Solusi KMP	Mekanisme
Pencarian pola tunggal	Cocok 100%	KMP secara alami dirancang untuk single pattern matching.

Efisiensi perulangan	Failure Function	Menghindari pergeseran pola yang tidak perlu dengan memanfaatkan informasi kecocokan sebelumnya.
Deteksi exact match	Border Array	Mengetahui seberapa jauh indeks harus melompat ke depan saat terjadi ketidakcocokan (mismatch).
Word boundary	Post-processing filter	Memastikan hasil pencarian berada di batas kata (bukan bagian dari substring kata lain).
Performance $O(n+m)$	Linear scanning	Terjamin memenuhi kompleksitas waktu $O(n+m)$ untuk satu kali proses pembacaan (single pass).

2. Algoritma Boyer-Moore

Elemen Masalah	Solusi Boyer-Moore	Mekanisme
Pencarian pola tunggal	Cocok 100%	Boyer-Moore merupakan alternatif efisien untuk single pattern matching.
Efisiensi perulangan rata-rata	Bad Character Rule	Melewati karakter teks yang sama sekali tidak ada di dalam pola pencarian.
Pola panjang	Good Suffix Rule	Melakukan pergeseran indeks yang lebih besar ketika pola kata kunci yang dicari semakin panjang.
Worst-case handling	Fallback linear scan	Mengantisipasi kasus terburuk $O(n*m)$ yang jarang terjadi pada teks nyata dengan pemindaian linier cadangan.
Performance $O(n/m)$	Right-to-left scanning	Melakukan pencocokan dari kanan ke kiri, sehingga rata-rata waktu pencarian bisa lebih cepat daripada KMP.

3. Algoritma Aho-Corasick

Elemen Masalah	Solusi Aho-Corasick	Mekanisme
Pencarian banyak pola sekaligus	Struktur Data Trie (Automaton)	Membangun state machine dari sekumpulan kata kunci

		(kamus), memungkinkan pencarian banyak pola secara bersamaan hanya dalam satu kali pemindaian teks.
Transisi saat terjadi ketidakcocokan	Failure Links	Saat terjadi mismatch, algoritma tidak mengulang pencarian dari awal teks (tanpa backtracking), melainkan berpindah melalui failure link ke awalan (prefix) pola lain yang memiliki kecocokan terpanjang.
Mendeteksi kecocokan yang tumpang tindih	Output/Dictionary Links	Memastikan tidak ada pola yang terlewat. Jika sebuah pola yang cocok ternyata mengandung sub-pola lain (misalnya mencari "he" dan "she"), output link akan mendeteksi keduanya secara instan.
Performa pada kasus teks panjang	Waktu Linier $O(n + m + z)$	Waktu pencarian berbanding lurus dengan panjang teks (n), total panjang semua pola pencarian (m), dan total jumlah temuan (z). Sangat stabil untuk teks berjumlah masif.

4. Algoritma Rabin-Karp

Elemen Masalah	Solusi Rabin-Karp	Mekanisme
Perbandingan string yang memakan waktu	Fungsi Hash (Hashing)	Mengubah pola pencarian dan substring pada teks menjadi nilai numerik (hash). Sistem membandingkan nilai angka ini terlebih dahulu alih-alih mengecek kecocokan per huruf.
Efisiensi perhitungan saat bergeser	Rolling Hash (Sliding Window)	Menghitung hash pada jendela teks berikutnya dalam waktu konstan $O(1)$ dengan cara matematis: mengurangi bobot karakter pertama yang keluar dari jendela dan menambahkan bobot karakter baru yang masuk.
Menghindari hasil positif palsu (Kolisi)	Spurious Hit Fallback	Karena dua kata berbeda bisa memiliki nilai hash yang sama (kolisi), algoritma tetap melakukan verifikasi karakter demi karakter setiap kali nilai hash

		dinyatakan cocok untuk memastikan kecocokan 100%.
Pencarian beberapa pola berukuran sama	Hash Set Matching	Nilai hash dari banyak pola disimpan di dalam sebuah Set. Saat rolling hash teks berjalan, algoritma cukup mencocokkan nilainya ke dalam Set tersebut, memungkinkannya menemukan banyak pola sekaligus.

3.3 Arsitektur Extension

Arsitektur Extension terdiri dari beberapa komponen utama yang saling berkomunikasi untuk melakukan deteksi dan penyorotan kata kunci pada halaman web. Pada sisi antarmuka, terdapat Popup UI yang berfungsi sebagai tampilan interaktif bagi pengguna untuk memilih algoritma pencocokan yang digunakan serta melihat statistik hasil deteksi. Popup UI berkomunikasi dengan Background Service Worker, yaitu komponen yang menangani proses latar belakang, message passing, serta pengelolaan status ekstensi. Background Service Worker kemudian berinteraksi dengan Content Script, yang bertindak sebagai logika utama dan dijalankan langsung pada halaman web aktif.

Di dalam Content Script terdapat tiga modul utama, yaitu Scraper, Matcher, dan Renderer. Modul Scraper bertugas mengambil dan mengekstraksi data teks dari struktur DOM Tree. Data hasil ekstraksi kemudian diproses oleh modul Matcher untuk melakukan pencocokan terhadap daftar kata kunci menggunakan algoritma tertentu. Proses pencocokan ini memanfaatkan Keywords Database, yang terdiri dari file keyword.txt sebagai daftar kata kunci dan homoglyph.txt sebagai pemetaan karakter serupa untuk menangani variasi penulisan. Setelah ditemukan kecocokan, modul Renderer bertugas memberikan visual feedback dengan menampilkan hasil deteksi secara langsung pada halaman, misalnya melalui proses highlighting teks.

3.4 Fitur-Fitur Fungsional yang Dibangun

Fitur	Deskripsi	Input	Output
Text Scraping	Extract semua teks dari DOM	DOM tree	Array of text nodes + content

Image Scraping	Extract teks dari gambar	IMG elements	Array of images + OCR text
Normalization	Normalize teks dan homoglyph	Raw text	Normalized text
KMP Matching	Exact match dengan KMP	Text + keywords	Array of offsets
Boyer-Moore Matching	Exact match dengan BM	Text + keywords	Array of offsets
Aho-Corasick Matching	Exact match dengan Aho-Corasick	Text + keywords	Array of ACResult
Rabin-Karp Matching	Exact match dengan Rabin-Karp	Text + keywords	Array of offsets
Fuzzy Matching	Approximate match dengan Levenshtein	Text + keywords	Array of offsets
Regex Matching	Pattern matching dengan regex	Text	Array of regex matches
Deduplication	Remove overlapping matches	All matches	Non-overlapping matches
Highlighting	Render <mark> tags	DOM + matches	Modified DOM with marks
Word Boundary Check	Ensure match is at word boundary	Text + offset	Boolean
Statistics Tracking	Track performance metrics	Matches + algorithm	Stats object
Blur Application	Apply CSS blur filter	Boolean enabled	Modified DOM styles

3.5 Contoh Ilustrasi Kasus

1. Exact Match dengan KMP

Pada skenario pertama, pengguna diasumsikan membuka sebuah situs berita yang memuat teks kalimat: "Website GACOR terbaru menawarkan bonus besar untuk pemain baru. Bergabunglah dengan GACOR99 sekarang dan dapatkan jackpot!". Sistem dikonfigurasi untuk mencari kata kunci ["GACOR", "GACER"] dengan memilih KMP sebagai algoritma utama.

Proses dimulai pada Tahap Ekstraksi dan Normalisasi, yaitu komponen Scraper mengisolasi dua text node dari DOM dan mengubah seluruh karakternya menjadi lowercase. Selanjutnya, Tahap Pencarian KMP mengeksekusi pembentukan array border untuk kata kunci "gacor". Berdasarkan tabel border tersebut, pencarian linier dilakukan pada kalimat pertama. Pada indeks ke-8, algoritma KMP mendeteksi

kecocokan eksak sepanjang 5 karakter. Sebelum hasil ini divalidasi, sistem melakukan Word Boundary Check untuk memastikan bahwa string tersebut diapit oleh spasi, sehingga statusnya dinyatakan valid.

Pencarian berlanjut pada kalimat kedua, di mana KMP kembali menemukan string "gacor" pada indeks ke-21 (di dalam kata "GACOR99"). Karena karakter setelah pencocokan adalah angka '9', Word Boundary Check tetap meloloskan string ini sebagai kecocokan yang valid. Sementara itu, pencarian untuk kata kunci "GACER" tidak menghasilkan apa-apa karena tidak ada karakter yang cocok sama persis.

2. Fuzzy Match dengan Typo

Pada skenario kedua, kondisinya adalah pengelola situs berusaha mengelabui mesin pencari dengan memodifikasi ejaan kata kunci, atau adanya faktor ketidaksengajaan (typo) oleh penulis. Teks target dalam kasus ini berbunyi: "Saya bermain di GACOR88 kemarin, dapat keuntungan 5 juta. Platform GACEOR terbaik di Asia Tenggara menurut saya." dengan kata kunci pencarian ["GACOR"].

Sistem mengawali proses dengan melakukan ekstraksi dan normalisasi teks menjadi huruf kecil. Pada saat Tahap KMP Matching dijalankan, string "gacor" pada kalimat pertama berhasil ditemukan pada posisi indeks ke-8. Namun, ketika melewati Word Boundary Check, sistem mendeteksi bahwa substring tersebut langsung diikuti oleh angka "88" tanpa adanya batas kata yang jelas, sehingga KMP membatalkan status kecocokannya. Pada kalimat kedua, KMP gagal total karena string "gaceor" memiliki susunan karakter yang berbeda.

Untuk mengatasi kegagalan tersebut, sistem mengalihkan pencarian ke Tahap Fuzzy Matching menggunakan perhitungan Levenshtein Distance. Sistem menghitung jarak edit antara kata kunci "gacor" dengan segmen teks yang dicurigai. Pada teks pertama ("gacor88"), algoritma menemukan kecocokan substring dengan jarak 0. Pada teks kedua ("gaceor"), algoritma mendeteksi adanya kelebihan huruf 'e', sehingga menghasilkan nilai distance sama dengan 1. Karena ambang batas (threshold) toleransi yang ditetapkan adalah 1, kedua temuan ini dinyatakan sebagai fuzzy match yang valid.

3. Homoglyph Detection

Skenario terakhir menguji ketahanan sistem terhadap teknik pengelabuan teks berbasis Unicode spoofing. Modus ini sering digunakan oleh situs ilegal untuk menghindari pemblokiran otomatis. Kalimat yang diuji tampak secara visual normal sebagai kata "GACOR", namun secara struktural karakter 'A' ditulis menggunakan kode karakter Cyrillic dan huruf 'O' diganti menggunakan karakter angka nol (GAC0R).

Saat teks ini masuk ke dalam sistem, algoritma tidak langsung melakukan pencarian. Pada Tahap Normalisasi, string mentah dialihkan ke komponen Homoglyph Mapper. Komponen ini memeriksa setiap Unicode penyusun karakter dan mencocokkannya dengan tabel pemetaan (homoglyph map) yang tersimpan di dalam ekstensi. Karakter 'C' Cyrillic secara otomatis dikonversi menjadi karakter 'C' ASCII standar, dan angka '0' (nol) dinormalisasi menjadi huruf 'O'.

Hasil akhir dari proses normalisasi ini mengembalikan string bersih berbunyi "gacor" dalam format huruf kecil standar. Ketika Tahap KMP Searching mengeksekusi string hasil normalisasi tersebut dengan kata kunci "GACOR", algoritma dapat menemukan kecocokan 100 persen secara eksak. Sistem kemudian mencatat bahwa kata kunci yang berhasil dideteksi adalah "GACOR", namun teks asli yang tertulis di dalam DOM adalah "GAC0R".

BAB 4

Implementasi dan Pengujian

4.1 Implementasi Program

Struktur Data Utama

A. Match

```
export interface Match {  
  keyword : string  
  matched : string  
  offset : number  
  length : number  
  algorithm : Algorithm  
  time : number  
}
```

Menyimpan informasi lengkap mengenai kata kunci yang berhasil dideteksi, termasuk teks aslinya, posisi karakter (offset), dan algoritma yang menemukannya.

B. TrieNode

```
export interface TrieNode {  
  children: Map<string, TrieNode>  
  isEnd: boolean  
  failureLink: TrieNode | null  
  patterns: string[]  
}
```

Struktur data pohon (Prefix Tree) yang digunakan oleh algoritma Aho-Corasick untuk melacak kecocokan banyak kata kunci secara simultan tanpa harus mengulang pembacaan teks.

C. StatsState

```
type StatsState = {  
  totalKeywordsFound: number  
  keywordCounts: Record<string, number>  
  keywordBuckets: Record<'exact' | 'fuzzy' | 'regex', Record<string,  
  number>>  
  perAlgorithm: Record<string, { matches: number; time: number }>  
}
```

Objek penyimpan status statistik agregat secara real-time (seperti total temuan dan waktu eksekusi tiap algoritma) yang nantinya dikirim dan dirender di antarmuka Popup.

Fungsi-Fungsi Utama

A. scrape() dan image_scrape()

```
export function scrape(): { node: Text; text: string }[] {
  const res: { node: Text, text: string }[] = []

  const text_scraper = document.createTreeWalker(
    document.body,
    NodeFilter.SHOW_TEXT,
    {
      acceptNode: (node) => {
        const parent = node.parentElement
        const tag = parent?.tagName
        if (tag === 'SCRIPT' || tag === 'STYLE') return
NodeFilter.FILTER_REJECT
        if (parent?.closest('highlight[data-judol-highlight]'))
return NodeFilter.FILTER_REJECT
        return NodeFilter.FILTER_ACCEPT
      }
    }
  )

  let node: Text
  while((node = text_scraper.nextNode() as Text)) {
    if (node.textContent.trim()) {
      res.push({node, text: node.textContent})
    }
  }

  return res
}

export async function image_scrape(): Promise<{img: HTMLImageElement,
text: string}>[]> {
  const img_scraper = document.createTreeWalker(
    document.body,
    NodeFilter.SHOW_ELEMENT,
    {
      acceptNode: (node) => {
        const tag = node.nodeName
        if(tag === 'IMG'){
          const img = node as HTMLImageElement
          // if (img.closest('highlight[data-judol-highlight]'))
return NodeFilter.FILTER_REJECT
          return NodeFilter.FILTER_ACCEPT
        } return NodeFilter.FILTER_SKIP
      }
    }
  )

  let filteredimg: HTMLImageElement[] = []
  let srcs: string[] = []

  let node: HTMLImageElement
  while((node = img_scraper.nextNode() as HTMLImageElement)) {
    // console.log(node.src, node.naturalWidth, node.complete)
    const loaded = await waitImage(node)
    if(!loaded) continue

    if(node.naturalWidth < 50 || node.naturalHeight < 50) continue
    const style = window.getComputedStyle(node)
    if(style.display === 'none' || style.visibility === 'hidden' ||
style.opacity === '0') continue

    filteredimg.push(node)
    srcs.push(node.src)
  }
}
```



```

    const fetched = await chrome.runtime.sendMessage({ type: 'SCAN_IMAGE',
    urls: srcs })

    const size = Math.ceil(fetched.length / 3)
    const fetched1 = fetched.slice(0, size)
    const fetched2 = fetched.slice(size, size*2)
    const fetched3 = fetched.slice(size*2)
    const [imgTexts1, imgTexts2, imgTexts3] = await Promise.all([
        scanImages(fetched1),
        scanImages(fetched2),
        scanImages(fetched3)
    ])
    const imgTexts = [...imgTexts1, ...imgTexts2, ...imgTexts3]

    let imgInfo: { img: HTMLImageElement, text: string }[] = []
    for(let i = 0; i < filteredimg.length; i++){
        imgInfo.push({img: filteredimg.at(i)!, text: imgTexts[i]!})
    }
    return imgInfo
}

```

Fungsi ini bertugas sebagai ekstraktor elemen dari struktur HTML menggunakan TreeWalker. Fungsi `scrape()` secara spesifik menyaring Node bertipe `TextNode`, secara otomatis mengabaikan tag yang tidak relevan seperti `<script>` dan `<style>`, serta teks yang sudah di-highlight sebelumnya. Sementara itu, `image_scrape()` mencari elemen `<img>`, memvalidasi ukuran dan visibilitasnya dengan mengabaikan gambar tersembunyi atau terlalu kecil, menunggu hingga gambar dimuat sepenuhnya, lalu mengirimkan daftar URL gambar tersebut ke Background Script agar diunduh secara latar belakang demi menghindari pemblokiran Cross-Origin Resource Sharing (CORS).

B. scanImages()

```

export async function scanImages(files: { bytes: number[], type:
string }[]): Promise<string[]> {
    const worker = await createWorker('eng+ind')
    await worker.setParameters({
        tesseract_pageseg_mode: PSM.SPARSE_TEXT,
        tesseract_char_whitelist: '',
    })
    let texts: string[] = []
    for(let file of files){
        if(!file){
            texts.push("")
            continue
        }
        try{
            const blob = bytesToBlob(file.bytes, file.type)
            const processedblob = await preprocess(blob)
            const { data: { text } } = await
worker.recognize(processedblob)
            texts.push(text.trim())
        } catch (e) {
            console.log("Failed in scanning: ", e)
            texts.push("")
        }
    }
}

```

```

    }
    return texts
  }
}

```

Fungsi ini mengubah data byte array gambar menjadi teks menggunakan engine OCR Tesseract.js. Sebelum proses pengenalan karakter dilakukan, gambar terlebih dahulu dimasukkan ke fungsi preprocess(). Di tahap pra-pemrosesan ini, gambar digambar ke atas elemen <canvas> HTML5 untuk dimanipulasi nilai pikselnya menjadi hitam-putih (binarization/grayscale) guna meningkatkan kontras teks. Gambar dengan resolusi rendah (lebar di bawah 600px) juga di-scale-up agar Tesseract dengan mode Sparse Text dapat membaca karakter di dalam gambar dengan tingkat akurasi yang lebih tinggi.

C. match()

```

export function match(text: string, keywords: string[], exactType: string)
: Match[] {
  const res: Match[] = []
  const lowerText = normalize(text.toLowerCase())
  const matchedKeywords = new Set<string>()

  if (exactType === 'aho-corasick') {
    const normalizedKeywords = keywords.map(kw =>
normalize(kw.toLowerCase()))
    const startAC = performance.now()
    const acMatches = ahoCorasick(lowerText, normalizedKeywords)
    const endAC = performance.now()

    acMatches.forEach(matchInfo => {
      if (hasWordBoundary(lowerText, matchInfo.offset,
matchInfo.length)) {
        // find original keyword before normalization
        const originalIndex =
normalizedKeywords.indexOf(matchInfo.keyword)
        const originalKeyword = originalIndex !== -1 ?
keywords[originalIndex]! : (matchInfo.keyword ?? '')

        res.push({
          keyword: originalKeyword,
          matched: text.substring(matchInfo.offset,
matchInfo.offset + matchInfo.length),
          offset: matchInfo.offset,
          length: matchInfo.length,
          algorithm: 'aho-corasick',
          time: endAC - startAC
        })
        matchedKeywords.add(originalKeyword)
      }
    })
  } else {
    let safeExactType = exactType?.trim().toLowerCase()
    let exactAlgorithm

    if (safeExactType === 'bm') {
      exactAlgorithm = bm
    } else if (safeExactType === 'rabin-karp') {
      exactAlgorithm = rabinKarp
    } else {

```

```

        exactAlgorithm = kmp
        safeExactType = 'kmp'
    }

    for (const keyword of keywords) {
        const normalizedKeyword = normalize(keyword.toLowerCase())

        const startexact = performance.now()
        const offsets = exactAlgorithm(lowerText,
normalizedKeyword).filter((offset) => hasWordBoundary(lowerText, offset,
normalizedKeyword.length))
        const endexact = performance.now()

        if (offsets.length > 0) {
            offsets.forEach(offset => {
                res.push({
                    keyword,
                    matched: text.substring(offset, offset +
keyword.length),
                    offset,
                    length: keyword.length,
                    algorithm: safeExactType as Match['algorithm'],
                    time: endexact - startexact
                })
            })
            matchedKeywords.add(keyword)
        }
    }
}

const startregex = performance.now()
const regexOffsets = regex(lowerText)
const endregex = performance.now()

if (regexOffsets.length > 0) {
    regexOffsets.forEach(offset => {
        const matched = text.substring(offset.offset, offset.offset +
offset.length)
        const keyword = matched.replace(/\d{2,3}$/u, '')

        res.push({
            keyword,
            matched,
            offset: offset.offset,
            length: offset.length,
            algorithm: 'regex',
            time: endregex - startregex
        })
    })
}

for (const keyword of keywords) {
    // if word has found in exact, don't do fuzzy
    if (matchedKeywords.has(keyword)) {
        continue
    }

    const normalizedKeyword = normalize(keyword.toLowerCase())
    const startfuzzy = performance.now()
    const fuzzyOffsets = fuzzy(lowerText, normalizedKeyword)
    const endfuzzy = performance.now()

    if (fuzzyOffsets.length > 0) {
        fuzzyOffsets.forEach(offset => {
            res.push({
                keyword,
                matched: text.substring(offset.offset, offset.offset +

```

```

offset.length),
    offset: offset.offset,
    length: offset.length,
    algorithm: 'fuzzy',
    time: endfuzzy - startfuzzy
  })
})
}
}

return removeOverlap(res)
}

```

Ini adalah fungsi orchestrator utama untuk pattern matching. Pertama, fungsi ini menormalkan teks yaitu menghapus trik manipulasi homoglyph. Kemudian, pencarian dilakukan dalam tiga tahapan. Tahap pertama adalah memanggil fungsi algoritma eksak yang dipilih seperti Aho-Corasick, KMP, atau Boyer-Moore dan mengumpulkan posisi indeks kemunculannya. Tahap kedua adalah menjalankan regex() untuk menangkap pola kata yang diikuti angka. Tahap ketiga, untuk kata kunci yang belum ditemukan, dijalankan algoritma Levenshtein Distance (fuzzy()) untuk menangkap typo. Di akhir eksekusi, fungsi removeOverlap() dipanggil untuk memastikan sebuah rentang teks hanya dicatat satu kali berdasarkan prioritas algoritma.

D. highlight() dan replaceImage()

```

export function highlight(node: Text, matches: Match[]) {
  const text = node.textContent ?? ''

  if (matches.length === 0) {
    return 0
  }

  const sortedMatches = [...matches].sort((a, b) => a.offset - b.offset)

  const fragment = document.createDocumentFragment()
  let lastIndex = 0
  let highlighted = 0
  const marks: HTMLElement[] = []

  for (const match of sortedMatches) {
    const start = match.offset
    const end = match.offset + match.length

    // Skip invalid or overlapping matches
    if (start < lastIndex || start < 0 || end > text.length) {
      continue
    }

    // Add text before the match
    fragment.append(document.createTextNode(text.slice(lastIndex,
start)))

    // Add highlighted match
    const mark = document.createElement('mark')

```

```

        mark.dataset.judolHighlight = 'true'
        mark.dataset.keyword = match.keyword
        mark.dataset.algorithm = match.algorithm
        mark.dataset.time = match.time.toString()
        mark.dataset.foundIndex = highlighted.toString()
        mark.textContent = text.slice(start, end)
        marks.push(mark)

        fragment.append(mark)

        lastIndex = end
        highlighted += 1
    }

    for (const mark of marks) {
        mark.dataset.count = highlighted.toString()
    }

    fragment.append(document.createTextNode(text.slice(lastIndex)))
    node.replaceWith(fragment)

    return highlighted
}

export function replaceImage(img: HTMLImageElement, src: string){
    img.removeAttribute("srcset")
    img.removeAttribute("sizes")

    img.src = src
    img.srcset = src

    img.loading = "eager"
}

```

Fungsi ini bertugas melakukan manipulasi Document Object Model (DOM). Fungsi `highlight()` menggunakan objek `DocumentFragment` untuk memecah teks asli menjadi beberapa bagian dan menyisipkan elemen penanda `<mark>` berwarna kuning tepat di koordinat indeks teks yang melanggar, lengkap dengan data attributes seperti jenis algoritma dan waktu untuk merender tooltip. Untuk elemen gambar, `replaceImage()` akan menghapus atribut responsif (`srcset`) dan mengganti sumber gambar utama (`src`) dengan URL placeholder penyensoran. Selanjutnya, `flagImage()` akan membungkus gambar tersebut ke dalam sebuah `<div relative>` dan menambahkan lapisan overlay peringatan visual tepat di atas gambar.

E. `scan()`

```

async function scan(algorithm?: string) {
    let selectedAlgorithm = algorithm

    if (!selectedAlgorithm) {
        const storage = await
chrome.storage.local.get('selectedAlgorithm') as { selectedAlgorithm?:
string }
        selectedAlgorithm = storage.selectedAlgorithm || 'kmp'
    }
}

```

```

clearHighlights()
resetStats()

console.log('    Scraping text...')
const scraped = scrape()
console.log(scraped, 'scraped text')
console.log('    Scraping image...')
const scrapedimg = await image_scrape()
const highlighted = []

console.log('Scraping successful, scanning...')

type ScrapedItem = { node?: Text, img?: HTMLImageElement, type: 'text'
| 'image', text: string }
const ScrapedItems = []
const censoredimg = []

for (const { node, text } of scraped) {
    ScrapedItems.push({ node: node, text: text, type: 'text' as const
})
}

for (const { img, text } of scrapedimg) {
    ScrapedItems.push({ img: img, text: text, type: 'image' as const
})
}

const sorted: ScrapedItem[] = ScrapedItems.sort((a, b) => {
    const nodeA = a.type === 'text' ? a.node : a.img
    const nodeB = b.type === 'text' ? b.node : b.img
    const position = nodeA!.compareDocumentPosition(nodeB!)
    if (position & Node.DOCUMENT_POSITION_FOLLOWING) return -1 // b
comes after a
    if (position & Node.DOCUMENT_POSITION_PRECEDING) return 1 // b
comes before a
    return 0
})

for (const { node, img, type, text } of sorted) {
    const matches = match(text, TEXT_POOL, selectedAlgorithm!)

    if (matches.length > 0) {
        if (type === 'text'){
            const count = highlight(node!, matches)

            recordStats(matches, count)
            updateStat()

            highlighted.push({
                text: node!.textContent ?? '',
                matches, count
            })
        } else if (type === 'image'){
            censoredimg.push({img, text, matches})
            replaceImage(img!, imagePlaceholder)
            flagImage(img!)
        }
    }
    await new Promise(r => setTimeout(r, 0))
}

updateStat()

return { highlighted, censoredimg }
}

```

Prosedur utama yang mengintegrasikan seluruh alur kerja sistem dalam satu siklus pemindaian. Fungsi ini dimulai dengan membaca konfigurasi dari storage lokal yaitu pilihan algoritma. Fungsi ini memanggil modul scraper untuk mengumpulkan teks dan hasil OCR gambar. Semua elemen yang terkumpul kemudian diurutkan berdasarkan posisinya secara hierarkis di dokumen HTML (Node.DOCUMENT_POSITION_FOLLOWING). Fungsi ini lalu mengiterasi setiap elemen, memanggil match(), lalu memicu fungsi manipulasi DOM (highlight atau sensor gambar). Terakhir, fungsi ini merekam statistik eksekusi ke memori (telemetry).

Alur Kerja Utama

1. Initialization

Saat halaman web atau content script termuat, program mengaktifkan observer untuk blur layar jika fitur perlindungan aktif, menyiapkan kamus kata kunci (keyword.txt), dan membangun referensi normalisasi homoglyph.

2. Triggering

Pengguna menekan tombol "Rescan" atau mengubah opsi algoritma di UI Extension (Popup). Popup mengirimkan pesan bertipe SCAN atau SET_ALGORITHM ke Content Script.

3. Scraping and Preprocessing

Scraper memindai seluruh teks (TextNode) dan gambar di halaman web. URL gambar dikirim ke Background Script, diunduh, lalu dikembalikan ke Content Script untuk dibaca teksnya menggunakan Tesseract OCR.

4. Matching Pipeline

Untuk setiap potong teks dari HTML maupun OCR gambar, program melakukan normalisasi karakter. Kemudian Matching Engine beraksi secara bertahap:

- a. Mencari kecocokan presisi (Exact) menggunakan algoritma pilihan.
- b. Mencari kecocokan berbasis pola menggunakan Regex.
- c. Jika kata kunci belum ditemukan, mencari kemiripan teks mentolerir kesalahan ketik (Fuzzy/Levenshtein).

d. Menghapus hasil temuan yang saling overlap

5. DOM Injection

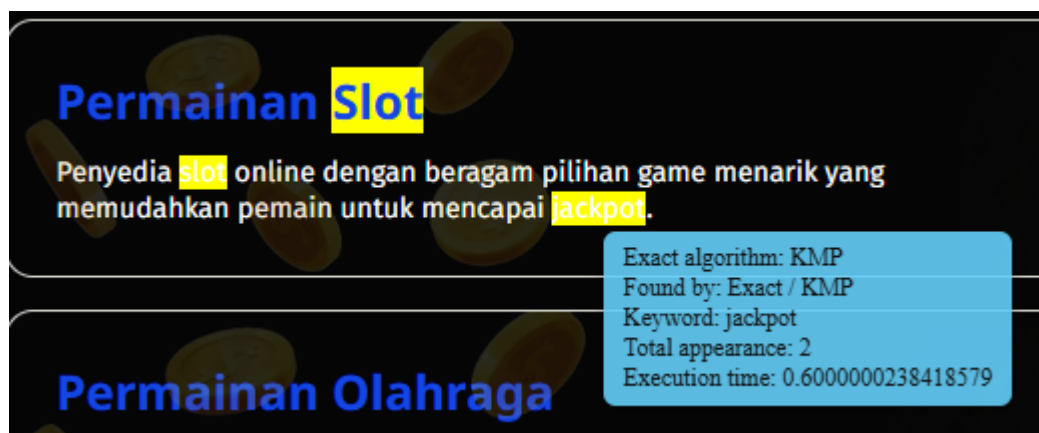
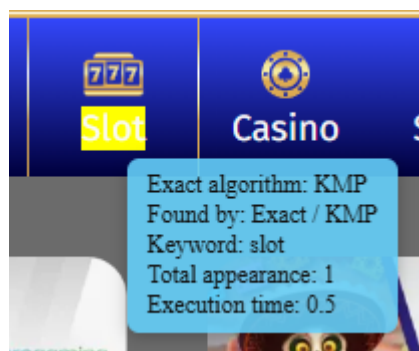
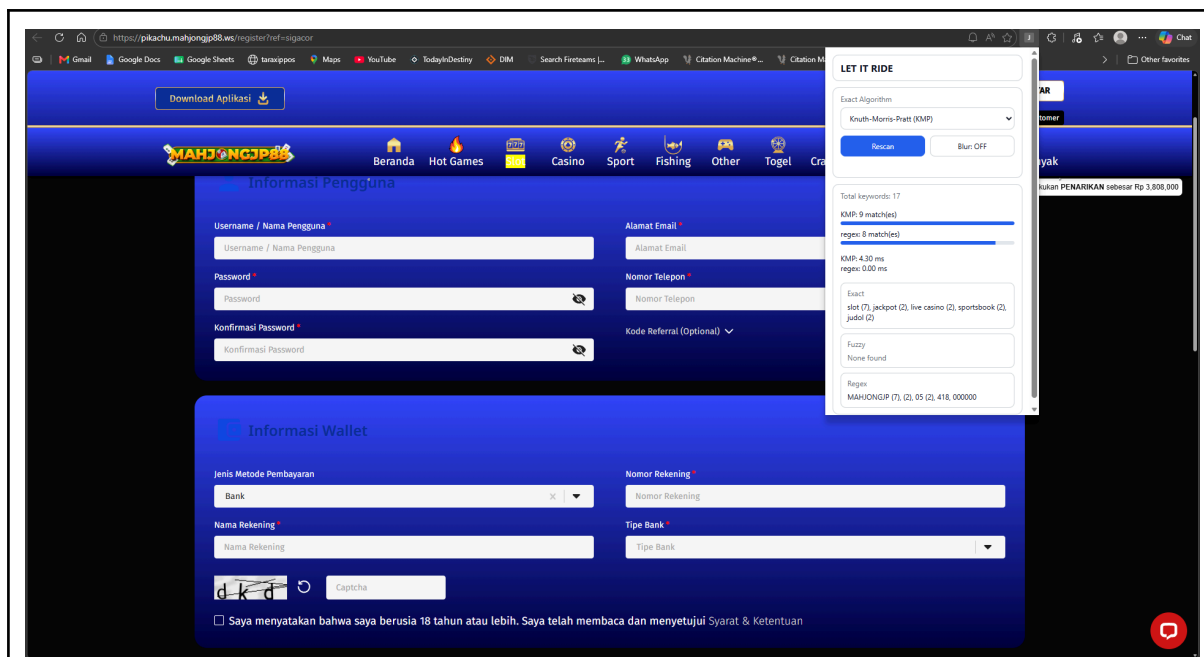
Berdasarkan data offset dan length yang ditemukan, teks di halaman dibungkus dengan <mark> kuning. Gambar yang terbukti mengandung teks terlarang akan ditimpa source-nya. Statistik komputasi dicatat ke StatsState.

4.2 Cara Penggunaan Extension

1. Clone githup repository
2. Masuk ke folder judol-extension
3. bun install
4. bun run dev untuk melakukan build
5. Buka browser (misalnya Google Chrome).
6. Masuk ke halaman chrome://extensions/
7. Aktifkan opsi Developer Mode.
8. Tekan tombol Load Unpacked.
9. Pilih folder `dist/` hasil build.

4.3 Testing

Link: MAHJONGJP88 - Link Trusted Situs Game Gacor Terpercaya & Terlengkap Hari Ini
Testing pada kasus Exact Matching Algoritma KMP
Hasil



Link: [MAHJONGJP88 - Link Trusted Situs Game Gacor Terpercaya & Terlengkap Hari Ini](https://pikachu.mahjongjp88.ws/register?ref=sagecor)

Testing pada kasus

Exact Matching Algoritma AHO CORASIC

Hasil

The screenshot displays a web application interface for 'MAHJONGP8'. The main content area is titled 'Buat Akun' (Create Account) and contains two sections: 'Informasi Pengguna' (User Information) and 'Informasi Wallet' (Wallet Information). The 'Informasi Pengguna' section includes fields for Username / Nama Pengguna, Password, Konfirmasi Password, Alamat Email, and Nomor Telepon. The 'Informasi Wallet' section includes fields for Jenis Metode Pembayaran (Bank), Nama Rekening, and Tipe Bank. A search results overlay titled 'LET IT RIDE' is visible on the right side of the screen. It shows the search results for the keyword 'jackpot' using the Aho-Corasic algorithm. The results indicate 6 matches, with a total appearance of 2 and an execution time of 0.5999999940395355. The search results also show the keyword 'jackpot' and its matches: 'slot (3), jackpot, live casino, sportsbook'.

agam pilihan game menarik yang
capai **jackpot**.

Exact algorithm: Aho-Corasic
Found by: Exact / Aho-Corasic
Keyword: jackpot
Total appearance: 2
Execution time: 0.5999999940395355

Link: Common Search: Slot99

Testing pada kasus
Regex

Hasil

Deep dive into **slot99**

- slot99.com
- slot99 link alter
- slot99 link
- slot99 link terbaru
- slot99 login
- slot99 free play
- slot99 game
- slot99 guide

pgslot99 สล็อตเว็บตรง สล็อตแตกง่าย สล็อต99

slot99 Found by: Regex
Keyword: pgslot
Total appearance: 1
Execution time: 0

YouTube • **pgslot99**
869 views · Oct 8, 2024

Regex hanya berlaku pada <kata><2-3 angka tanpa spasi>

Link: Local index.html

Testing pada kasus
Fuzzy Matching

Hasil

Matcher samples

Exact sample: judi online.

Fuzzy sample: slt gac0r, depozt pulsa, and bonus n3w memb3r.

Regex sample: Gacor999, HOKI88, and other digit-extended variants.

More regex sample: Awesome999 should also be detected.

depozt pulsa, and bonus n3w memb3r.

, HOKI88, and other digit-extended variants.

esome99

Found by: Fuzzy
Keyword: deposit pulsa
Total appearance: 3
Execution time: 0.30000001192092896

Link: [SLOT88KU : Login Agen Situs Slot Online Gacor & Slot777 Resmi Bisa Dimainkan Kapan Saja](#)

Testing pada kasus
OCR - Boyer Moore

Hasil

Sebelum Scanning

SITUS RESMI SLOT ONLINE GACOR TERPILIH



SLOT88KU

SLOT88KU SITUS SLOT TERGACOR Q

-  **PENCARIAN CEPAT**
-  **AKURAT & TERPERCAYA**
-  **JACKPOT TERUPDATE**
-  **AMAN & TERJAMIN**

CARI JACKPOT?
SERAHKAN KE SLOT88KU
TEMUKAN. MAIN. MENANG!

LOGIN **DAFTAR**

Setelah Scanning





Terindikasi judol bozo 🎰🎲

LOGIN

DAFTAR

SLOT ONLINE

LIVE CASINO

BOLA ONLINE

PRAGMATIC PG SOFT

EVOLUTION GAMING

SBOBET SABA

KLAIM BONUS NEW MEMBER DISINI!!

Copyright © SLOT88KU | BUKAN SEO

Exact Algorithm

Boyer-Moore (BM)

Rescan

Blur: OFF

Total keywords: 7

BM: 6 match(es)

regex: 1 match(es)

BM: 2.20 ms

regex: 0.00 ms

Exact

situs resmi slot, gacor, slot, live casino, pg soft, klaim bonus

Fuzzy

None found

Regex

SLOT

LIVE CASINO

BOLA ONLINE

EVOLUTION

SBOBET SABA

KLAIM BONUS NEW MEMBER DISINI!!

Exact algorithm: Boyer-Moore
Found by: Exact / Boyer-Moore
Keyword: live casino
Total appearance: 1
Execution time: 0

Lampiran

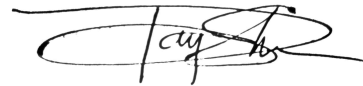
Github repository: [Arbane557/Tubes3 LET-IT-RIDE](#)

Youtube: [How To Stop Gambling Addiction 101 - STIMA TUBES 3 LET IT RIDE](#)

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Josh Reinhart Zidik



Raysha Erviandika Putra



Muhammad Naufal Romi Annafi

No	Poin	Ya	Tidak
1	Extension berhasil di-build dan di-load tanpa kesalahan pada chromium browser dan dikembangkan dengan TypeScript	✓	
2	KMP dan Boyer-Moore diimplementasikan from scratch	✓	
3	Regex menhandle format <kata><angka> dan berbagai edge case	✓	
4	Pencarian KMP & BM membaca keyword.txt secara iteratif dan tidak menggunakan built-in search function atau library eksternal	✓	

5	Exact matching dan fuzzy matching berjalan benar		
6	Elemen DOM terdeteksi diberi highlight dan terhapus saat rescanning	✓	
7	Tooltip muncul saat hover dengan informasi keyword, algoritma, kemunculan, dan waktu eksekusi	✓	
8	Popup menampilkan statistik realtime (total keyword, perbandingan, waktu eksekusi, jumlah match)	✓	
9	[Bonus] Membuat Video	✓	
10	[Bonus] Implementasi Algoritma Aho-Corasick dan Rabin Karp	✓	
11	[Bonus] Implementasi Censorship / Blur Teks	✓	
12	[Bonus] Implementasi Optical Character Recognition pada Gambar	✓	